BDD SQL — Notes de cours (Séances 1 à 4)

BUT3 GEA — IUT de Saint-Denis

Ce document résume les notions vues en séances 1 à 4. C'est une référence pour réviser l'examen : il ne remplace pas les feuilles d'exercices.

Tous les documents du cours (notes, énoncés, bases de données) sont sur **https://github.com/frey-iutsd/sql-2026**.

Base de données relationnelle : l'idée de base

Une base de données relationnelle stocke des données dans des **tables**, un peu comme des feuilles Excel nommées, mais avec deux différences importantes :

- chaque **colonne** a un nom et un type fixé à l'avance (nombre entier, texte, etc.) ;
- une même table a toujours la même structure de colonnes, ligne après ligne.

On interroge une base de données avec le langage **SQL** (*Structured Query Language*), grâce à des requêtes qui permettent de retrouver rapidement des informations précises, même dans une table de plusieurs milliers de lignes — ce qui serait très laborieux à faire à la main dans Excel.

Les logiciels

SQLiteStudio

Installer SQLiteStudio chez soi. Téléchargez la version **3.4.21** (celle des salles de l'IUT) sur https://github.com/pawelsalawa/letos/releases : le fichier `SQLiteStudio-3.4.21-windows-x64-installer.exe` pour Windows, ou `SQLiteStudio-3.4.21-macos-x64-installer.dmg` pour Mac. Le logiciel a été renommé « Letos » à partir de sa version 4 ; pour ce cours, gardez la 3.4.21.

- Si Windows bloque l'installation : **Informations complémentaires**, puis **Exécuter quand même**.
- Si le Mac refuse d'ouvrir le logiciel : **clic droit → Ouvrir**, ou **Réglages Système → Confidentialité et sécurité → Ouvrir quand même**.

Utiliser SQLiteStudio.

- **Ouvrir une base** : **Ctrl+O** (sur Mac : **Cmd+O**).
- **Ouvrir l'éditeur SQL** : menu **Outils → Ouvrir l'éditeur SQL**, ou raccourci **Alt+E**.
- **Exécuter une requête** : cliquez sur le triangle bleu ▶, ou appuyez sur **F9**. SQLiteStudio exécute la requête sur laquelle se trouve votre curseur — pas besoin d'un éditeur différent par requête.
- **Formater une requête** (l'afficher proprement sur plusieurs lignes) : raccourci **Ctrl+T**.

> 💡 **Passer SQLiteStudio en français.** Si le logiciel est en anglais : **Tools → Open Configuration dialog**, puis à gauche **Look & feel**, puis dans la liste déroulante **Language**, choisissez *French*. Redémarrez SQLiteStudio pour que le changement prenne effet.

Dans le panneau de gauche, chaque table a deux onglets qui nous intéressent : **Structure** (la liste des colonnes) et **Données** (le contenu, ligne par ligne).

> 💡 **Plusieurs bases ouvertes en même temps.** Dans la barre d'outils de l'éditeur SQL, une liste déroulante indique sur quelle base la requête va s'exécuter. Si vous obtenez une erreur « no such table », vérifiez d'abord que c'est la bonne base qui est choisie.

Sans rien installer : sqliteonline.com

Si SQLiteStudio ne s'installe pas (sur Mac en particulier), **https://sqliteonline.com/** fait la même chose dans le navigateur :

1. Menu **File → Open DB**, puis choisissez votre fichier `.db` (chinook.db ou votre base individuelle).
2. Écrivez la requête dans la zone de texte, puis cliquez sur **Run**. **Attention : Run exécute tout ce qui est dans la zone de texte.** Pour n'exécuter qu'une commande, sélectionnez-la à la souris avant de cliquer sur Run (sinon, un `CREATE TABLE` ou un `INSERT` déjà exécuté est relancé, et donne une erreur ou une ligne en double).
3. Les modifications (`INSERT`, `UPDATE`…) restent dans le navigateur. Si vous rechargez la page, rouvrez votre fichier d'origine et réexécutez vos commandes : gardez-les donc dans votre fichier texte.

SELECT et FROM

La commande de base pour interroger une table :

```
SELECT * FROM customers;
```

`*` signifie « toutes les colonnes ». Pour n'afficher que certaines colonnes, on les liste séparées par des virgules :

```
SELECT FirstName, LastName, Country FROM customers;
```

Filtrer avec WHERE

Pour n'afficher que certaines lignes, on ajoute une condition :

```
SELECT * FROM customers WHERE Country = "Brazil";
```

Opérateurs de comparaison :

Opérateur	Signification
`=`	égal
`LIKE`	égal, insensible à la casse
`!=`	différent
`<`	inférieur à
`<=`	inférieur ou égal
`>`	supérieur à
`>=`	supérieur ou égal

`LIKE` se comporte comme `=`, sauf qu'il ne fait pas la différence entre majuscules et minuscules :

```
SELECT * FROM customers WHERE Firstname = "jack";
-- ne trouve pas "Jack" (majuscule)

SELECT * FROM customers WHERE Firstname LIKE "jack";
-- trouve aussi "Jack"
```

> 💡 **Apostrophes ou guillemets ?** Pour une valeur de texte, la norme SQL est l'**apostrophe simple** : `WHERE Country = 'Brazil'`. SQLiteStudio accepte aussi les guillemets doubles (`"Brazil"`, comme dans les premiers exemples de ce document), mais ceux-ci désignent normalement un *nom de colonne*, et certains outils (dont parfois sqliteonline) les refusent pour du texte. Prenez l'habitude des apostrophes simples.

Les jokers % et _

Utilisés avec `LIKE`, deux caractères spéciaux permettent de chercher un *motif* plutôt qu'une valeur exacte :

- `%` représente une suite de caractères quelconque, **y compris vide**.
- `_` représente **exactement un** caractère.

```sql
SELECT * FROM customers WHERE Firstname LIKE "a%";
-- commence par "a"

SELECT * FROM customers WHERE Firstname LIKE "%a%";
-- contient un "a"

SELECT * FROM customers WHERE Firstname LIKE "_a%";
-- "a" en deuxième position

SELECT * FROM customers WHERE Email LIKE "%@gmail.com";
-- se termine par "@gmail.com"
```

Combiner des conditions : AND, OR, NOT, IN

```sql
SELECT * FROM invoices WHERE Total >= 10 AND Total <= 20;

SELECT *
FROM customers
WHERE Country = "France"
   OR Country = "Germany";
```

`IN` évite d'enchaîner plusieurs `OR` sur la même colonne :

```sql
SELECT *
FROM customers
WHERE Country IN ("France", "Germany", "UK");

-- équivalent à :
SELECT *
FROM customers
WHERE Country = "France"
   OR Country = "Germany"
   OR Country = "UK";
```

`NOT IN` fait l'inverse (exclut une liste de valeurs) :

```sql
SELECT *
FROM customers
WHERE Country NOT IN ("USA", "Canada");
```

> 💡 **AND et OR ensemble : mettez des parenthèses.** `AND` est évalué avant `OR`. `WHERE Pays = "France" OR Pays = "Espagne" AND Solde > 100` se lit `Pays = "France" OR (Pays = "Espagne" AND Solde > 100)`. Les parenthèses rendent l'intention explicite.

DISTINCT : supprimer les répétitions

Une colonne peut contenir des valeurs répétées (par exemple, plusieurs factures pour la même ville). `DISTINCT` ne garde qu'une occurrence de chaque valeur :

```sql
SELECT DISTINCT BillingCountry FROM invoices;
```

ORDER BY : trier les résultats

```sql
SELECT DISTINCT BillingCity, BillingCountry
FROM invoices
ORDER BY BillingCountry ASC;
```

`ASC` (croissant) est l'ordre par défaut ; `DESC` inverse l'ordre (décroissant).

Colonnes calculées et AS

On peut afficher le résultat d'un calcul plutôt qu'une colonne brute, et lui donner un nom avec `AS` :

```sql
SELECT Milliseconds / 1000 AS Seconds FROM tracks;
```

Ceci ne modifie pas la table : le calcul est fait uniquement pour l'affichage du résultat.

Arrondir avec `ROUND`. La division ci-dessus donne un résultat entier (SQLite tronque, sans arrondir, quand les deux valeurs sont des entiers). Pour un arrondi contrôlé, avec un nombre choisi de décimales, on utilise `ROUND(valeur, décimales)` :

```
SELECT ROUND(Milliseconds / 1000.0, 2) AS Seconds FROM tracks;
-- le ".0" force un calcul décimal (pas entier)
-- ROUND(..., 2) arrondit à 2 chiffres après la virgule
```

> **Division entière vs décimale.** En SQLite, `7 / 2` vaut `3` (division entière, le reste est perdu), alors que `7.0 / 2` ou `7 / 2.0` vaut `3.5`. Dès qu'un des deux nombres a une virgule (même `.0`), le calcul devient décimal. C'est une source d'erreur fréquente : pensez-y si un résultat calculé semble « arrondi à l'entier » sans que vous l'ayez demandé.

> **Virgule ou point ?** En SQL, les nombres décimaux s'écrivent avec un **point** : `1.2`. La virgule sépare deux colonnes : `montant * 1,2` donne deux colonnes, `montant * 1` et `2`.

Clé primaire, clé étrangère

Une **clé primaire** (*primary key*) identifie chaque ligne d'une table de façon unique (par exemple, `CustomerId` dans la table `customers`).

Une **clé étrangère** (*foreign key*) est une colonne d'une table qui référence la clé primaire d'une autre table, pour représenter un lien entre les deux. Par exemple, dans `invoices`, la colonne `CustomerId` référence `customers.CustomerId` : chaque facture est associée à un client précis.

C'est la même idée qu'une relation entre deux feuilles dans Access, ou qu'un `VLOOKUP` / `RECHERCHEV` entre deux feuilles Excel via un identifiant commun.

Dans SQLiteStudio, l'onglet **Structure** d'une table indique sa clé primaire (une petite icône de clé, dans la colonne correspondante), et l'onglet **Contraintes** liste ses clés étrangères.

INNER JOIN : combiner deux tables

Une jointure permet de combiner les lignes de deux tables lorsqu'une colonne de l'une correspond à une colonne de l'autre — typiquement, une clé étrangère et la clé primaire qu'elle référence.

```
SELECT customers.FirstName, customers.LastName, invoices.Total
FROM invoices
JOIN customers ON invoices.CustomerId = customers.CustomerId;
```

`JOIN` (ou `INNER JOIN`, équivalent) ne garde que les lignes qui trouvent une correspondance dans les deux tables. On peut ajouter un `WHERE` après la jointure, exactement comme d'habitude :

```
SELECT customers.FirstName, customers.LastName, invoices.Total
FROM invoices
JOIN customers ON invoices.CustomerId = customers.CustomerId
WHERE invoices.Total > 15;
```

> **Pourquoi préfixer `customers.FirstName` ?** Dès qu'une requête combine plusieurs tables, deux colonnes de tables différentes peuvent porter le même nom. Préfixer par le nom de la table (`table.colonne`) lève toute ambiguïté, et c'est une bonne habitude à prendre dès qu'une jointure est en jeu.

> **N'oubliez pas le `ON`.** Sans condition `ON`, SQLite associe *chaque* ligne de la première table à *chaque* ligne de la seconde : 412 factures × 59 clients = 24 308 lignes, qui n'ont aucun sens.

Anatomie d’une requête

Voici, annotée, la requête de l’exemple ci-dessus — pour visualiser d’un coup le rôle de chaque partie :



Anatomie d’une requête : chaque clause répond à une question différente.

Fonctions d’agrégation : COUNT, SUM, AVG, MIN, MAX *(séance 3)*

Une **fonction d’agrégation** prend toute une colonne et la résume en **une seule valeur** — comme `=NB()`, `=SOMME()` ou `=MOYENNE()` appliquées à une colonne entière dans Excel.

Fonction	Résultat
`COUNT(*)`	nombre de lignes
`SUM(colonne)`	somme des valeurs
`AVG(colonne)`	moyenne
`MIN(colonne)`	plus petite valeur
`MAX(colonne)`	plus grande valeur

```sql
SELECT COUNT(*) FROM customers;
-- 59 : le nombre de clients

SELECT COUNT(*) AS nb_factures,
       SUM(Total) AS ca_total,
       ROUND(AVG(Total), 2) AS panier_moyen,
       MIN(Total) AS plus_petite,
       MAX(Total) AS plus_grande
FROM invoices;
-- une seule ligne : 412 | 2328.6 | 5.65 | 0.99 | 25.86
```

Avec un `WHERE`, le calcul ne porte que sur les lignes gardées :

```sql
SELECT SUM(Total) FROM invoices WHERE BillingCountry = "France";
-- 195.1 : le chiffre d'affaires réalisé en France
```

> 💡 **COUNT(*) ou COUNT(colonne) ?** `COUNT(*)` compte toutes les lignes. `COUNT(colonne)` ne compte que les lignes où cette colonne est remplie : les cases vides (`NULL`) sont ignorées. Par exemple, `SELECT COUNT(*), COUNT(Company) FROM customers;` donne `59 | 10` : seuls 10 clients ont une entreprise renseignée.

GROUP BY : un résultat par groupe *(séance 3)*

Pour obtenir un résultat **par pays** (et non pour toute la table), on regroupe les lignes avec `GROUP BY` : la fonction d’agrégation est alors calculée séparément pour chaque groupe.

```sql
SELECT BillingCountry, SUM(Total) AS ca_pays
FROM invoices
GROUP BY BillingCountry
ORDER BY ca_pays DESC;
-- une ligne par pays : USA 523.06, Canada 303.96, France 195.1,
...
```

C’est l’équivalent d’un **tableau croisé dynamique** (TCD) dans Excel : la colonne du `GROUP BY` est mise en *lignes*, et la fonction d’agrégation donne les *valeurs*.

On peut calculer plusieurs agrégats par groupe, et regrouper selon plusieurs colonnes :

```sql
SELECT BillingCountry, BillingCity, COUNT(*) AS nb_factures,
SUM(Total) AS ca
FROM invoices
GROUP BY BillingCountry, BillingCity;
-- une ligne par couple (pays, ville)
```

> 💡 **La règle à retenir.** Dans le `SELECT` d'une requête avec `GROUP BY`, on ne met que **les colonnes du `GROUP BY`** et **des fonctions d'agrégation**. Par exemple, `SELECT BillingCountry, BillingCity, SUM(Total) FROM invoices GROUP BY BillingCountry;` n'a pas de sens : il y a plusieurs villes par pays, laquelle afficher ? SQLite ne signale pas d'erreur et en prend une au hasard — d'autres logiciels SQL refusent la requête. Le résultat est faux dans les deux cas.

GROUP BY après une jointure *(séance 3)*

On peut regrouper le résultat d'une jointure. Par exemple, le montant total dépensé par chaque client :

```sql
SELECT customers.CustomerId, customers.LastName,
       COUNT(*) AS nb_factures,
       SUM(invoices.Total) AS total_depense
FROM invoices
JOIN customers
  ON invoices.CustomerId = customers.CustomerId
GROUP BY customers.CustomerId, customers.LastName
ORDER BY total_depense DESC;
```

> 💡 **Regrouper par l'identifiant, pas seulement par le nom.** Deux clients différents peuvent porter le même nom. Si on regroupe par nom seulement, leurs factures sont additionnées comme s'il s'agissait d'une seule personne. La clé primaire (`CustomerId`) est unique : c'est elle qui garantit « un groupe = un client ».

HAVING : filtrer les groupes *(séance 3)*

`HAVING` sert à ne garder que certains **groupes**, selon le résultat d'une fonction d'agrégation :

```sql
SELECT Country, COUNT(*) AS nb_clients
FROM customers
GROUP BY Country
HAVING COUNT(*) > 5;
-- pays de plus de 5 clients : USA (13), Canada (8)
```

La différence avec `WHERE` :

- `WHERE` filtre les **lignes**, **avant** de faire les groupes ;
- `HAVING` filtre les **groupes**, **après** les avoir faits.

On ne peut donc pas écrire `WHERE COUNT(*) > 5` (erreur *misuse of aggregate*). Les deux peuvent apparaître dans la même requête :

```sql
SELECT BillingCountry, COUNT(*) AS nb_grosses_factures
FROM invoices
WHERE Total > 10       -- factures de plus de 10 $
GROUP BY BillingCountry -- regroupées par pays
HAVING COUNT(*) >= 5;  -- pays qui en ont au moins 5
```

Jointure sur trois tables *(séance 4)*

> 💡 **Le schéma de chinook** (toutes les tables et leurs liens) : https://www.sqlitetutorial.net/sqlite-sample-database/ . Chaque trait relie une clé étrangère à la clé primaire qu'elle référence : c'est la carte à suivre pour écrire une jointure.

Quand l'information est répartie sur trois tables, on enchaîne les `JOIN`, **un par lien** entre deux tables. Par exemple, dans chinook, une piste appartient à un album (`tracks.AlbumId`), et un album à un artiste (`albums.ArtistId`). Pour afficher chaque piste avec son album et son artiste :

```sql
SELECT tracks.Name, albums.Title, artists.Name
FROM tracks
JOIN albums  ON tracks.AlbumId = albums.AlbumId
JOIN artists ON albums.ArtistId = artists.ArtistId;
```

`WHERE`, `GROUP BY` et `HAVING` s'utilisent ensuite exactement comme avec une seule jointure. Par exemple, les artistes qui ont au moins 50 pistes de plus de 5 minutes :

```sql
SELECT artists.Name, COUNT(*) AS nb_pistes_longues
FROM tracks
JOIN albums  ON tracks.AlbumId = albums.AlbumId
JOIN artists ON albums.ArtistId = artists.ArtistId
WHERE tracks.Milliseconds > 300000
GROUP BY artists.ArtistId, artists.Name
HAVING COUNT(*) >= 50
ORDER BY nb_pistes_longues DESC;
-- Iron Maiden 117, Lost 91, Metallica 75, Led Zeppelin 54, The Office 53
```

Pour écrire une telle requête, partez de la table qui contient ce que vous **comptez ou filtrez** (ici les pistes), puis suivez les clés étrangères, une par une, jusqu'à la table qui contient ce que vous voulez **afficher** (ici le nom de l'artiste).

Créer une table : CREATE TABLE *(séance 4)*

```sql
CREATE TABLE fournisseurs (
    id   INTEGER PRIMARY KEY,
    nom  TEXT NOT NULL,
    pays TEXT
);
```

Pour chaque colonne, on donne son **nom**, son **type**, et éventuellement des **contraintes** :

Type	Contenu
`INTEGER`	nombre entier
`REAL`	nombre à virgule
`TEXT`	texte

Contrainte	Effet
`PRIMARY KEY`	clé primaire : deux lignes ne peuvent pas avoir la même valeur
`NOT NULL`	la case ne peut pas rester vide
`REFERENCES autre_table(colonne)`	clé étrangère : la valeur doit correspondre à une ligne de l'autre table

Pour **voir la structure** d'une table existante, dans SQLiteStudio comme dans sqliteonline :

```sql
PRAGMA table_info(clients);
-- une ligne par colonne : nom, type, notnull (1 = obligatoire),
pk (1 = clé primaire)

SELECT sql FROM sqlite_master WHERE name = 'clients';
-- la commande CREATE TABLE qui a créé la table (onglet DDL de
SQLiteStudio)
```

(*DDL*, pour *Data Definition Language*, désigne les commandes qui définissent la **structure** des tables : `CREATE TABLE`, `ALTER TABLE`. Les commandes qui travaillent sur les **données** elles-mêmes, `INSERT`, `UPDATE`, `DELETE` et aussi `SELECT`, forment le *DML*, *Data Manipulation Language*.)

Pour ajouter une colonne à une table qui existe déjà :

```sql
ALTER TABLE produits ADD COLUMN fournisseur_id INTEGER REFERENCES
fournisseurs(id);
-- la nouvelle colonne est vide (NULL) pour toutes les lignes
existantes
```

Ajouter des lignes : INSERT *(séance 4)*

On liste les colonnes, puis les valeurs **dans le même ordre**. Les textes sont entre guillemets, pas les nombres.

```sql
INSERT INTO fournisseurs (id, nom, pays) VALUES (1, 'Bureau
Plus', 'France');

-- plusieurs lignes d'un coup :
INSERT INTO fournisseurs (id, nom, pays) VALUES
    (2, 'TechDistri', 'Allemagne'),
    (3, 'Maison & Co', 'Espagne');

-- sans donner l'id : SQLite prend le numéro suivant
INSERT INTO fournisseurs (nom, pays) VALUES ('Papeterie du Nord',
'Belgique');
```

Les contraintes protègent la table : insérer deux fois `id = 3` donne l'erreur *UNIQUE constraint failed*, oublier une colonne `NOT NULL` donne *NOT NULL constraint failed*.

Modifier et supprimer : UPDATE, DELETE *(séance 4)*

```sql
-- modifier une colonne pour les lignes choisies
UPDATE produits SET fournisseur_id = 1 WHERE id IN (2, 5, 6);

-- avec un calcul : +5 % sur les produits du fournisseur 1
UPDATE produits SET prix = ROUND(prix * 1.05, 2) WHERE
fournisseur_id = 1;

-- supprimer les lignes choisies
DELETE FROM commandes WHERE montant < 30;
```

> 💡 **Toujours un WHERE, et le tester d'abord.** Sans `WHERE`, `UPDATE` et `DELETE` s'appliquent à **toutes** les lignes de la table, et il n'y a pas de bouton « annuler ». Avant d'exécuter la commande, lancez `SELECT * FROM … WHERE …` avec la même condition pour voir quelles lignes seront touchées.

L'ordre des clauses

L'ordre d'écriture d'une requête est **fixe** :

```sql
SELECT ...      -- quelles colonnes, quels calculs ?
FROM ...        -- quelle table ?
JOIN ... ON ... -- quelle autre table, reliée comment ? (un JOIN
par table ajoutée)
WHERE ...       -- quelles lignes garder ?
GROUP BY ...    -- quels groupes faire ?
HAVING ...      -- quels groupes garder ?
ORDER BY ...    -- dans quel ordre afficher ?
```

Seuls `SELECT` et `FROM` sont obligatoires ; les autres clauses sont facultatives, mais quand elles sont présentes, elles apparaissent dans cet ordre-là.

Forme générale des commandes qui modifient la base *(séance 4)*

Les crochets `[ ]` indiquent une partie facultative ; ils ne s'écrivent pas.

```sql
-- structure (DDL)
CREATE TABLE nom_table (
    colonne1 TYPE [PRIMARY KEY],
    colonne2 TYPE [NOT NULL],
    colonne3 TYPE [REFERENCES autre_table(id)]
);
ALTER TABLE nom_table ADD COLUMN colonne TYPE [REFERENCES
autre_table(id)];

-- données (DML)
INSERT INTO nom_table (col1, col2, ...) VALUES (valeur1, valeur2,
...), (valeur1, valeur2, ...);
UPDATE nom_table SET col1 = expression, col2 = expression [WHERE
condition];
DELETE FROM nom_table [WHERE condition];
```

Ces commandes sont plus simples que `SELECT` : la seule clause facultative est `WHERE`, toujours à la fin. Elle s'écrit exactement comme dans un `SELECT` (`AND`, `OR`, `IN`, `LIKE`…).

Commande	À retenir
`INSERT`	les valeurs dans le **même ordre** que les colonnes ; plusieurs lignes séparées par des virgules
`UPDATE`	`SET` avant `WHERE` ; plusieurs colonnes séparées par des **virgules** : `SET prix = 10, stock = 5` (et non `SET prix = 10 AND stock = 5`, qui ne fait pas ce qu'on veut)
`DELETE`	pas de liste de colonnes : on supprime toujours des **lignes entières** (`DELETE FROM commandes WHERE …`, et non `DELETE montant FROM …`)
`UPDATE`, `DELETE`	sans `WHERE` : **toutes** les lignes de la table